Objects and Classes – Part 2

Definitions we learnt so far.

- What is an object?
- What is a class?
- Abstraction
- Mutators **(commonly called as setters)**
- Accessors **(commonly called as getters)**
- Private Helpers
- Initialization
- Constructor
- Constructor overloading

Constructor Overloading

A class creator can ***overload*** a constructor by defining multiple constructors differing in parameter types. When an object is created with the new operator, arguments can be passed to the constructor. The constructor with matching parameters will be called.

```
public Inventory(){
    FruitType = "some fruit";
    FruitQuantity = -1;
    FruitPrice = -1;
}

public Inventory(String Fruitname){
    FruitType = Fruitname;
    FruitQuantity = -1;
    FruitPrice = -1;
}

public Inventory(String Fruitname, int NewItems){
    FruitType = Fruitname;
    FruitQuantity = NewItems;
    FruitPrice = -1;
}
```

Default Constructor

Class definition

```
public class Inventory {

    private String FruitType;
    private int FruitQuantity;
    private float FruitPrice;
```

Default constructor

```
public Inventory(){
    FruitType = "NULL";
    FruitQuantity = 0;
    FruitPrice = 0;
}
```

Java would create the default constructor explicitly.

```
Inventory Fruit1 = new Inventory();
Fruit1.printDetails();
```

There are 0 null available at $0.0

Adding a Constructor

Class definition

```
public class Inventory {

    private String FruitType;
    private int FruitQuantity;
    private float FruitPrice;
```

User defined constructor

```
public Inventory(String FruitName, int
NewItems, float NewCost){
    FruitType = FruitName;
    FruitQuantity = NewItems;
    FruitPrice = NewCost;
}
```

Java would not create the default constructor explicitly.

```
Inventory Fruit1 = new Inventory();
Fruit1.printDetails();
```

ERROR

ALWAYS HAVE A DEFAULT CONSTRUTOR

```
public Inventory(){
    FruitType = "some fruit";
    FruitQuantity = -1;
    FruitPrice = -1;
}

public Inventory(String Fruitname){
    FruitType = Fruitname;
    FruitQuantity = -1;
    FruitPrice = -1;
}

public Inventory(String Fruitname, int NewItems){
    FruitType = Fruitname;
    FruitQuantity = NewItems;
    FruitPrice = -1;
}
```

Not defining any constructor is essentially the same as defining a constructor with no arguments.

JAVA WILL NOT CREATE DEFAULT CONSTRUCTOR if there is at least one user written constructor. WILL RESULT IN COMPILER ERROR

New Definitions

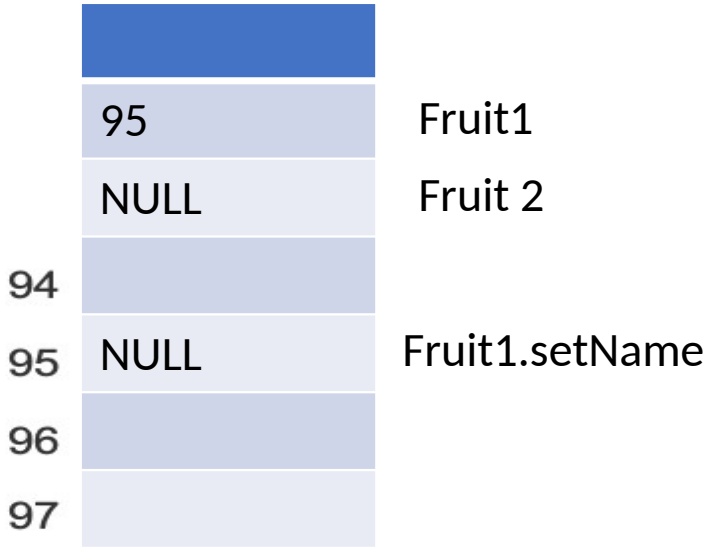
- References
- Multiple Object References
- ‘this’ implicit parameter
- Wrapper classes vs Primitive classes
- Unit testing classes

Objects and References

A ***reference*** is a variable type that refers to an object. A reference may be thought of as storing the memory address of an object. Variables of a class data type (and array types, discussed elsewhere) are reference variables.

```
Inventory Fruit1 = new Inventory();

Inventory Fruit2;
```



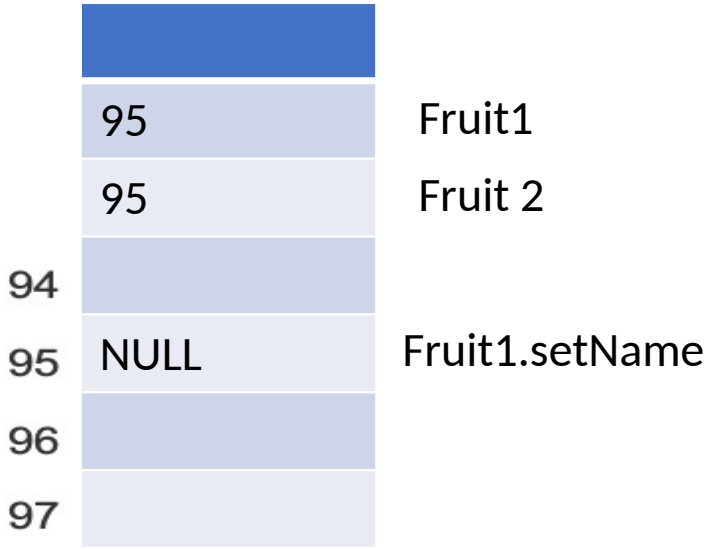
Multiple References

Two or more reference variables may refer to the same object, as illustrated below.

```
Inventory Fruit1 = new Inventory();

Inventory Fruit2;

Fruit2=Fruit1;
```



‘this’ parameter – Implicit parameter

Within a member method, the implicitly-passed object reference is accessible via the keyword ***this***. In particular, a class member can be accessed as this.classMember. The "." is the member access operator.

```
public void setName(String FruitType) {
    this.FruitType = FruitType;                    //Mutator
}

public void setQuantity(int FruitQuantity) {
    this.FruitQuantity = FruitQuantity;                //Mutator
}
```

Using this makes clear that a class member is being accessed and is essential if a field member and parameter have the same identifier. In the example below, this is necessary to differentiate between the field member sideLength and the parameter sideLength.

‘this’ parameter on a constructor

From within a constructor, you can also use the `this` keyword to call another constructor in the same class. Doing so is called an *explicit constructor invocation*. Here's another `Rectangle` class, with a different implementation from the one in the Objects section.

```
public class Rectangle {
    private int x, y;
    private int width, height;

    public Rectangle() {
        this(0, 0, 1, 1);
    }
    public Rectangle(int width, int height) {
        this(0, 0, width, height);
    }
    public Rectangle(int x, int y, int width, int height) {
        this.x = x;
        this.y = y;
        this.width = width;
        this.height = height;
    }
    ...
}
```

toString()

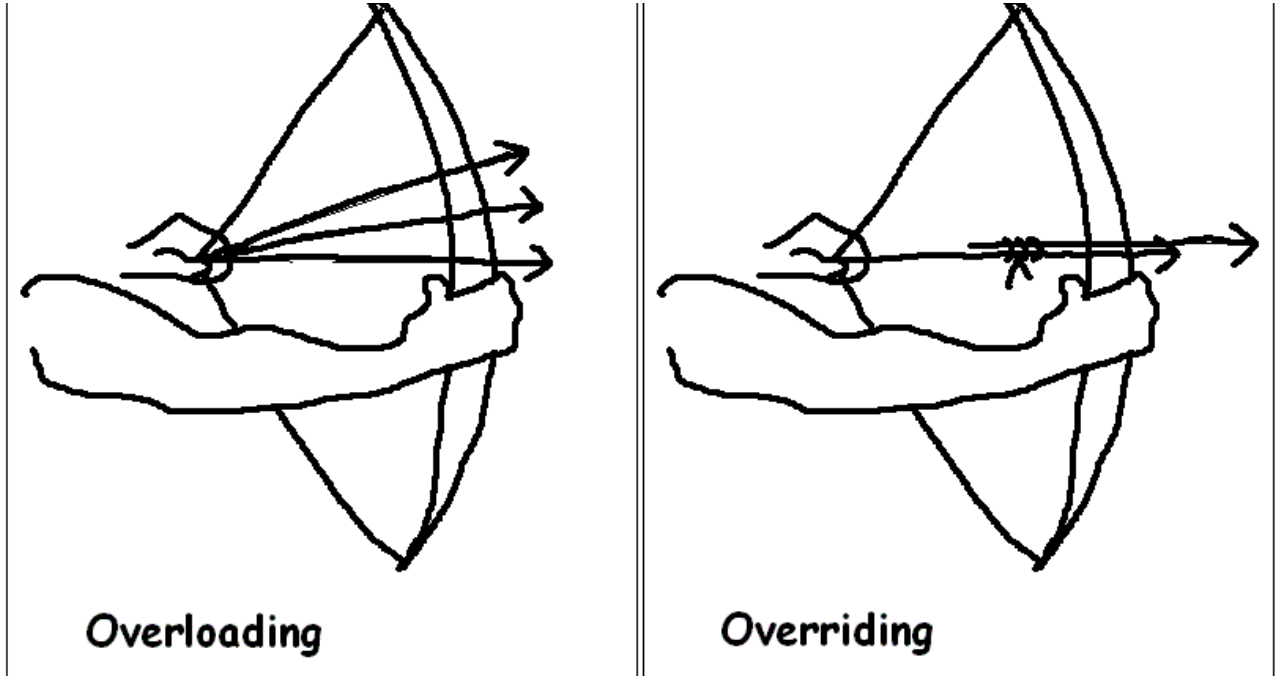
Java toString() Method

- If you want to represent any object as a string, toString() method comes into existence.
- The toString() method returns the String representation of the object.
- If you print any object, Java compiler internally invokes the toString() method on the object. So overriding the toString() method, returns the desired output, it can be the state of an object etc. depending on your implementation.

Advantage of Java toString() method

- By overriding the toString() method of the Object class, we can return values of the object, so we don't need to write much code.

Overloading vs Overriding



Overloading: methods with the same name but have different parameter types.

Overriding: a subclass method takes precedence over a superclass method with the same name and parameter types but probably with different implementations.

Primitive and reference types

- **Primitive type**: a variable directly stores the data for that variable type, such as `int, double, char.`

- **Reference type**: a variable refer to an instance of a class, also known as an object, such as `String, Integer, FileInputStream.`

Wrapper Classes

Wrapper classes are built-in reference types that augment the primitive types.

E.g. The `Integer` data type is a built-in class in Java that augments the `int` primitive type.

```
Integer a = 10; //declares an Integer reference variable named a that refers to
                an instance of the Integer class, or Integer object.
```

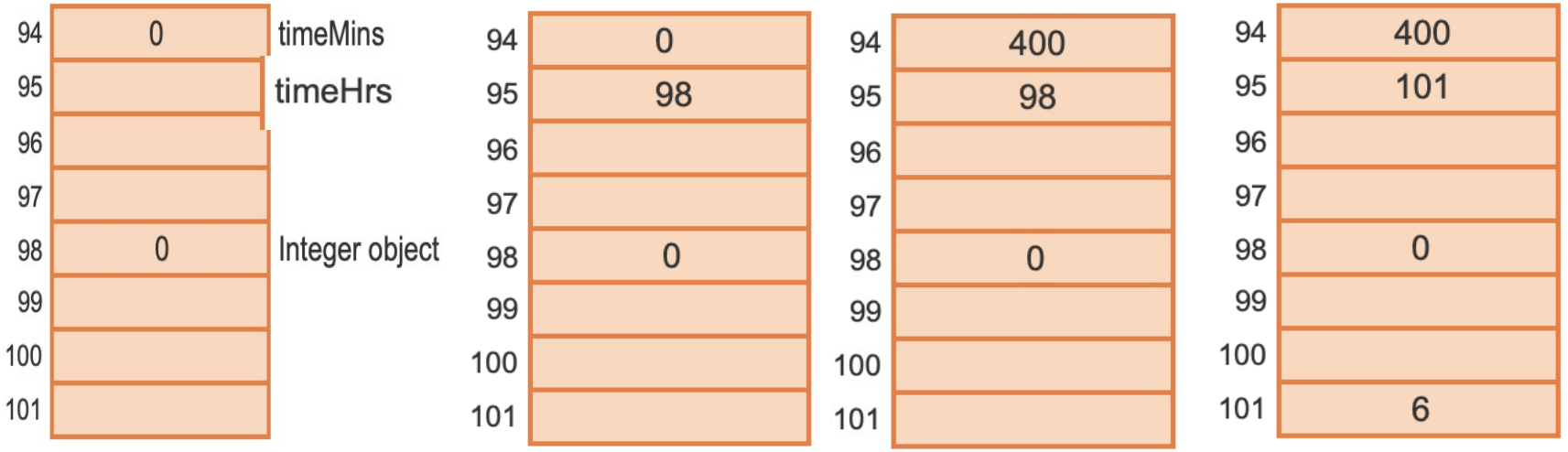
Reference type	Associated primitive type
Character	char
Integer	int
Double	double
Boolean	boolean
Long	long

Wrapper classes are immutable

- A wrapper class object (as well as a `String` object) is **immutable**, meaning a programmer cannot change the object via methods or variable assignments after object creation.

```
int timeMins = 0;
Integer timeHrs = 0;

timeMins = 400;
timeHrs = timeMins / 60;
```



Wrapper Classes

Wrapper classes have inbuilt methods

```
Integer.MIN_VALUE

Integer.MAX_VALUE
```

Wrapper classes cannot be compared using == or !=

```
Integer a = 5; Integer b = 5;

a == b (not correct)

a.equals(b)    (correct)
```

Converting to primitive types

Given:

```
Integer num1 = 14;
Double num2 = 6.7643;
Double num3 = 5.6e12;
```

intValue()	Returns the value of the wrapper class object as a primitive int value, type casting if necessary. `num2.intValue() // Returns 6`
doubleValue()	Returns the value of the wrapper class object as a primitive double value, type casting if necessary. `num1.doubleValue() // Returns 14.0`
longValue()	Returns the value of the wrapper class object as a primitive long value, type casting if necessary. `num3.longValue() // Returns 5600000000000`

Autoboxing and Unboxing

Autoboxing is the automatic conversion of primitive types to the corresponding wrapper classes.

```
Integer a = 5;
```

Unboxing is the automatic conversion of wrapper class objects to the corresponding primitive types.

```
int b = a;
```

Test bench

A ***testbench*** is a program whose job is to thoroughly test another program (or portion) via a series of input/output checks known as ***test cases***.

Unit testing means to create and run a testbench for a specific item (or "unit") like a method or a class.

Regression testing

Regression testing means to retest an item like a class anytime that item is changed; if previously-passed test cases fail, the item has "regressed".
A testbench should be maintained along with the item, to always be usable for regression testing.

Test bench (Example)

Class to test: StatsInfo.java

```java
public class StatsInfo {

   // Note: This class intentionally has errors

   private int num1;
   private int num2;

   public void setNum1(int numVal) {
      num1 = numVal;
   }

   public void setNum2(int numVal) {
      num2 = numVal;
   }

   public int getNum1() {
      return num1;
   }

   public int getNum2() {
      return num1;
   }

   public int getAverage() {
      return num1 + num2 / 2;
   }
}
```

Testbench: StatsInfoTest.java

```java
public class StatsInfoTest {
   public static void main(String[] args) {
      StatsInfo testData = new StatsInfo();

      // Typical testbench tests more thoroughly

      System.out.println("Beginning tests.");

      // Check set/get num1
      testData.setNum1(100);
      if (testData.getNum1() != 100) {
         System.out.println("   FAILED set/get num1");
      }

      // Check set/get num2
      testData.setNum2(50);
      if (testData.getNum2() != 50) {
         System.out.println("   FAILED set/get num2");
      }

      // Check getAverage()
      testData.setNum1(10);
      testData.setNum2(20);
      if (testData.getAverage() != 15) {
         System.out.println("   FAILED GetAverage for 10, 20");
      }

      testData.setNum1(-10);
      testData.setNum2(0);
      if (testData.getAverage() != -5) {
         System.out.println("   FAILED GetAverage for -10, 0");
      }

      System.out.println("Tests complete.");
   }
}
```

In-Class Activity – Coding a Vending Machine

Write a code for VendingMachine with snacks and drinks as objects and that performs the following operations:

- Create a new item on the machine
- Purchase drinks : input the quantity
- Restock drinks : input number of bottles
- Reports inventory : Snack/Drink name +

Your code should have 3 classes:

A class to create the object. (Add constructors, getters and setters)

A main class to create instances of object

A test class to test the objects.